

DSA Day 4: Dynamic arrays (malloc -> Python list)

Book: Data Structures Through C in Depth - Ch2 §2.3 · pp 27-34 | Code in Python

Overview

C uses malloc/realloc/free to grow arrays. Python's list does this automatically by over-allocating and doubling capacity, giving amortised $O(1)$ append.

Key concepts to master

- Heap allocation, sizeof, free (C)
- Over-allocation & capacity doubling
- Amortised analysis of append
- Memory leaks / dangling pointers (C)

Python implementation

```
class DynamicArray:
    def __init__(self):
        self.cap=1; self.n=0; self.a=[None]*self.cap
    def append(self,x):
        if self.n==self.cap:
            self.cap*=2; self.a=self.a[:] + [None]*self.cap
        self.a[self.n]=x; self.n+=1
```

Complexity

append amortised $O(1)$; resize $O(n)$ but rare.

Common pitfalls

- Forgetting free() in C leaks memory
- Thinking every append is $O(n)$

Practice

- Print capacity on each doubling
- Explain amortised $O(1)$

